

Exame Modelo 2 — Paradigmas de Programação — Época de Recurso 2025/2026

Instituição	P.PORTO — Escola Superior de Tecnologia e Gestão
Tipo de Prova	Exame Escrito — Época de Recurso
Curso	Licenciatura em Engenharia Informática / Licenciatura em Segurança Informática em Redes de Computadores
Unidade Curricular	Paradigmas de Programação
Ano Letivo	2025/2026
Duração	2 horas

Observações

- Não é permitida a consulta.
- Não são permitidas questões relativas à Parte 2. Sempre que considerarem necessário, os alunos devem assumir os pressupostos que entenderem adequados, indicando-os explicitamente na resolução.

Parte 1

Pergunta 1 (1,5 valores)

Explique detalhadamente o conceito de exceções em Java. Distinga entre exceções verificadas (*checked exceptions*) e exceções não verificadas (*unchecked exceptions*). Descreva o funcionamento do mecanismo `try-catch-finally` e explique quando é adequado criar exceções personalizadas. Ilustre com um exemplo prático que demonstre a criação e o lançamento de uma exceção personalizada.

Pergunta 2 (1,5 valores)

Explique o conceito de construtores em Java, descrevendo as suas características e regras de definição. Distinga entre construtor por defeito, construtor parametrizado e encadeamento de construtores (com `this(...)` e `super(...)`). Esclareça o que acontece quando uma classe não define nenhum construtor explicitamente. Ilustre com exemplos de código.

Pergunta 3 (1,5 valores)

Explique detalhadamente o conceito de tipos enumerados (`enum`) em Java. Descreva as vantagens de utilizar `enum` em vez de constantes inteiras ou Strings para representar conjuntos fixos de valores. Demonstre como um `enum` pode ter atributos, construtores e métodos. Ilustre com um exemplo prático.

Pergunta 4 (1,5 valores)

Explique as diferenças entre composição e herança em Java, descrevendo quando cada mecanismo é mais adequado. Justifique por que razão a composição é frequentemente preferida em relação à herança no desenvolvimento de software e forneça um exemplo prático que demonstre a composição entre duas classes.

Parte 2

1. Considere as seguintes interfaces `Vehicle` e `Route`. A interface `Vehicle` descreve as operações possíveis que definem o contrato de um veículo utilizado na recolha de bens de uma instituição de ajuda humanitária. A interface `Route` define o contrato de uma rota de recolha.

```
public interface Vehicle {  
    String getCode();  
    ItemType getSupplyType();  
    double getMaxCapacity();  
}
```

```
public interface Route {  
    Vehicle getVehicle();  
    void addAidBox(AidBox aidBox) throws RouteException;  
    AidBox removeAidBox(AidBox aidBox) throws RouteException;  
    AidBox[] getRoute();  
    boolean equals(Object obj);  
}
```

```
public interface AidBox {  
    String getCode();  
    String getZone();  
    Container[] getContainers();  
}
```

```
public interface Container {  
    ItemType getType();  
    double getCapacity();  
    Measurement getLastMeasurement();  
}
```

```
public interface Measurement {  
    double getValue();  
}
```

Pergunta 1a (3 valores)

Considere a interface `Route` que representa uma rota de recolha de bens. Implemente a interface numa classe denominada `RouteImpl`. A rota deve ser associada a um veículo no momento da criação. Deve conter um array de `AidBox` com capacidade máxima de 10 posições. O método `addAidBox` deve lançar `RouteException` se a `AidBox` for nula ou se já existir na rota. O método `removeAidBox` deve lançar `RouteException` se a `AidBox` não for encontrada na rota. Para o método `equals`, duas rotas são iguais se estiverem associadas ao mesmo veículo (comparando os códigos dos veículos).

Pergunta 1b (2 valores)

Desenvolva o código necessário para testar a classe implementada (por exemplo, no contexto de um método `main`). Apresente um exemplo de teste para cada método implementado.

1. Considere a existência de uma classe `CollectionManagerImpl` que implemente a interface `CollectionManager`.

```
public interface CollectionManager {  
    double getTotalCollectedByType(IIInstitution inst, ItemType type);  
}
```

Pergunta 2a (4 valores)

Implemente os seguintes métodos na classe `CollectionManagerImpl`:

```
double getContainerLoad(Container container);
```

- Este método deve devolver o valor da última medição do contentor. Se o contentor não possuir medição ou for nulo, deve devolver 0.

```
boolean isContainerFull(Container container, double threshold);
```

- Este método deve devolver `true` se a última medição do contentor for superior à percentagem `threshold` da sua capacidade. Se o contentor não possuir medição, deve devolver `false`.

Pergunta 2b (5 valores)

Na classe `CollectionManagerImpl`, implemente o método `getTotalCollectedByType`.

Regras a considerar:

- O método deve percorrer todas as `AidBoxes` devolvidas pelo método `getAidBoxes()` da interface `IIInstitution`.
- Para cada `AidBox`, deve percorrer os seus contentores e somar os valores de carga (utilizando `getContainerLoad`) de todos os contentores cujo tipo corresponda ao `ItemType` recebido como

argumento.

- Apenas devem ser considerados contentores que estejam cheios (utilizando `isContainerFull` com threshold de 75%).
- O método deve devolver o total de carga recolhida para o tipo especificado.

Excertos de Código Fornecidos

```
public class InstitutionImpl implements IInstitution {  
    (...)  
    private AidBox[] aidBoxes;  
    private int numberOfAidBoxes;  
    private Vehicle[] vehicles;  
    private int numberOfVehicles;  
    (...)  
  
    public AidBox[] getAidBoxes() {  
        (...)  
    }  
  
    public Vehicle[] getVehicles() {  
        (...)  
    }  
}
```